Flexion XS

Rapport d’activité

Sommaire

Flexion XS.....1

Sommaire.....2

Tâche N°1.....3

Tâche N°2.....4

Tâche N°3.....5

Tâche N°4.....6

Tâche N°5.....7

Tâche N°6.....11

Tâche N°8.....16

ÉCOLE NATIONALE D'INGÉNIEURS DE BREST

Tâche N°1

Gestion du travail de groupe

Intitulé complet de la tâche:

COEFFICIENT DE LA TACHE: **3**

- **planning** avec affectation des tâches et %
- **POSTER** A3 mettant en avant les liens interdisciplinaires
- **Schéma fonctionnel** global
- **organisation** du travail collaboratif (partage de documents, archivage)



LE CORFF Louis

70%



MADEC Mael

0%



BEUGUIA DADEM Janvier

15%



MEVEL Yoann

15%

1.Création des planning
Présenter et découper les fonctions associées à la tâche.

2. Création du poster A3
Créé le poster A3 mettant en avant les liens interdisciplinaires.

3. Schéma fonctionnel
Réactualisation des documents existant pour se conformer au câblage.

4. Création d'un tutoriel
Création d'un tutoriel pour expliquer le fonctionnement de notre projet et des outils mis en place

5. Création d'une vidéo pour montrer le fonctionnement
Création d'une vidéo pour montrer le fonctionnement du projet et de nos outils

6. Perspectives d'évolution
Création d'un fichier contenant une liste d'idée d'amélioration ainsi que certains de nos problème pour le prochain groupe

Tâche N°2

Simulation du comportement vibratoire (étude libre, harmonique et modale)

Intitulé complet de la tâche:

- Simulation de la réponse dynamique du dispositif (comprenant l'étude de la cinématique complète).
- Programme de calcul
- Notice de calcul



1. Récupération des documents de Mr Helbert

Le jupyter notebook de Mr Helbert nous fournissait un signal test généré manuellement pour faire une analyse modale. Nous avons donc adapté les variables avec nos valeurs et nous avons créé des boucles des différentes fonctions pour trois modes et trois capteurs.

2. Analyse en régime libre

Sur notre système de réglette, nous installons plusieurs fixations pour accrocher l'accéléromètre et utiliser des pois pour maintenir une répartition des masses constante. Nous déplacerons l'accéléromètre sur les différentes fixations pour faire notre analyse en régime libre.

3.Analyse modale

Pour l'analyse en régime libre, on commence par le calcul des 3 réponses fréquentielles à partir des signaux temporels, puis pour chaque mode, on calcule la bande à 3 dB de l'amplitude dont on cherche la moyenne pour calculer les matrices modales puis on recommence pour chaque capteur. On pourra ainsi visualiser la pulsation, l'amortissement et les valeurs propres.
Pour finir, on calcule les matrices MSF et MAC.

Tâche N°3

Conception mécanique CAO et intégration physique

Intitulé complet de la tâche:

-Conception d'un excentrique avec réducteur
-Conception d'un excentrique sans réducteur
-Conception de support pour le moteur
-Remplacement des supports des accéléromètres.
- bilan et résolution des dysfonctionnements des impressions et des capteurs en condition de mesure.

COEFFICIENT DE LA TACHE: 4



LE CORFF Louis
50%



MADEC Mael
0%



BEUGUIA DADEM
Janvier
50%



MEVEL Yoann
0%

1. Conception en CAO

Lors de nos mesures à faible vitesse, nos valeurs reçues étaient composées en grande partie du déplacement causé par la présence d'un excentrique important en sortie de notre moteur, et nous ne pouvions pas observer les vibrations correctement. Pour palier à ce problème, nous avons conçu un nouvel excentrique avec une excentricité plus faible et ainsi ne pouvoir observer que les variations de vitesse dues à nos vibrations.
Après cela nous avons constaté que la vitesse en sortie du motoréducteur était trop faible et nous empêchait d'observer le second mode. Pour augmenter la vitesse d'excitation de la réglette nous avons donc enlevé le réducteur du moteur et conçu un nouvel excentrique. Et comme le support ne permettait de ne positionner qu'un motoréducteur, nous avons conçu un nouveau support pour le moteur en CAO.

Conception d'un support d'accéléromètre à la suite d'un changement d'orientation du capteur, en premier lieu pour obtenir une meilleure fréquence d'échantillonnage max. et ensuite pour ne pas déséquilibrer un côté lors de la vibration. La fixation se fait par encastrement avec une vis sans fin.

2. Imprimer les pièces

Impression 3D du nouveau support et des différents excentriques.

Impression 3D du support avec une 10aine de réalisations pour obtenir les meilleures pièces possibles. Des problèmes de tailles, de filetages et de sur-remplissages expliquent ce grand nombre.

3. Changer les pièces

Remplacements de nos pièces en vue des différents tests.

ÉCOLE NATIONALE D'INGÉNIEURS DE BREST

Tâche N°4

Taux de charge motorisation et réglage PID

COEFFICIENT DE LA TACHE: **1**

Intitulé complet de la tâche:

.A partir du modèle numérique électrodynamique, estimer le taux de charge du moteur dans le cas de fonctionnement le plus défavorable. Implanter un correcteur et vérifier par simulation le bon fonctionnement du système (consigne de vitesse)



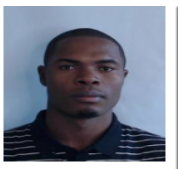
LE CORFF Louis

0%



MADEC Mael

100%



BEUGUIA DADEM Janvier

0%



MEVEL Yoann

0%

1. Calculer le taux de charge du moteur

Estimer le taux de charge du moteur en fonction du modèle numérique électrodynamique pour le scénario de fonctionnement le plus défavorable.

Une modélisation du moteur est réalisée dans le Jupyter Notebook fourni. Les paramètres utilisés sont issus de la datasheet et de quelques mesures, notamment d'inertie.

Le calcul de taux de charge est cependant lui plus complexe à réaliser dans le contexte de ce semestre : l'excentrique est redessiné en parallèle d'une réflexion sur le retrait du réducteur pour multiplier par 30 la vitesse (et donc les fréquences de vibrations) du moteur.

2. Régler le PID

Implémenter un correcteur PID et ajuster ses paramètres pour garantir un fonctionnement optimal du système, notamment en termes de consigne de vitesse.

Nous avons décidé de faire 2 choix pour ce qui est du PID :
La partie "Dérivateur" est utilisée et le moteur ne doit pas dépasser sa commande (càd du **No Overshoot**)
Un premier réglage de PID est implémenté par défaut dans le programme STM32 et s'est révélé fonctionnel dans son contrôle du moteur.

3. Perspectives de changements

Puisque le PID a été calculé dans une perspective où le taux de charge est faible, il n'est pas forcément adapté pour les hautes vitesses. C'est pour cela que le Jupyter Notebook existe : il permet de renseigner les nouvelles informations et d'adapter en conséquence le PID. Le réglage est heuristique (c'est à dire à l'œil) et s'est révélé fiable en adéquation avec le code STM32 fourni.

Tâche N°5

Conception électrique CAO et intégration physique

Intitulé complet de la tâche:

COEFFICIENT DE LA TACHE: **4**

- Changer les composants cramés
- Refaire les documents pour que la nomenclature correspondent à la réalité
- Tester les continuités
- Prendre en compte les erreurs de réalisations, de conceptions.
-Dépanner les nombreux problèmes qui sont apparus lors d'expérimentations ou suite à de l'usure.
-Remplacer des composants, des câbles ou des parties mécaniques pour permettre de continuer le développement du projet.
-Expérimenter des voies de pensées pour répondre à des problèmes et tenter d'améliorer/pallier la mauvaise condition de la maquette.



LE CORFF Louis

20%



MADEC Mael

0%



BEUGUIA DADEM Janvier

80%



MEVEL Yoann

0%

1 Prise en main du projet à travers la revue des archives et de l'étude des différents composants

Prise en main des archives et des vidéos présentent, et analyse plus poussée des composants pour comprendre le fonctionnement de la carte électrique,et de la maquette.

Le hacheur sert à varier la tension en entrée du moteur à l'aide d'un PWM, ce qui permet de piloter le moteur en vitesse. On utilise dans notre cas le moteur B, donc les pins OUT 3 et 4. Les pins input 3 et 4 permettent de contrôler le sens de rotation du moteur. Il faut connecter le hacheur aux +12V/GND afin qu'il puisse être fonctionnel.

2. Remise en état de la machine

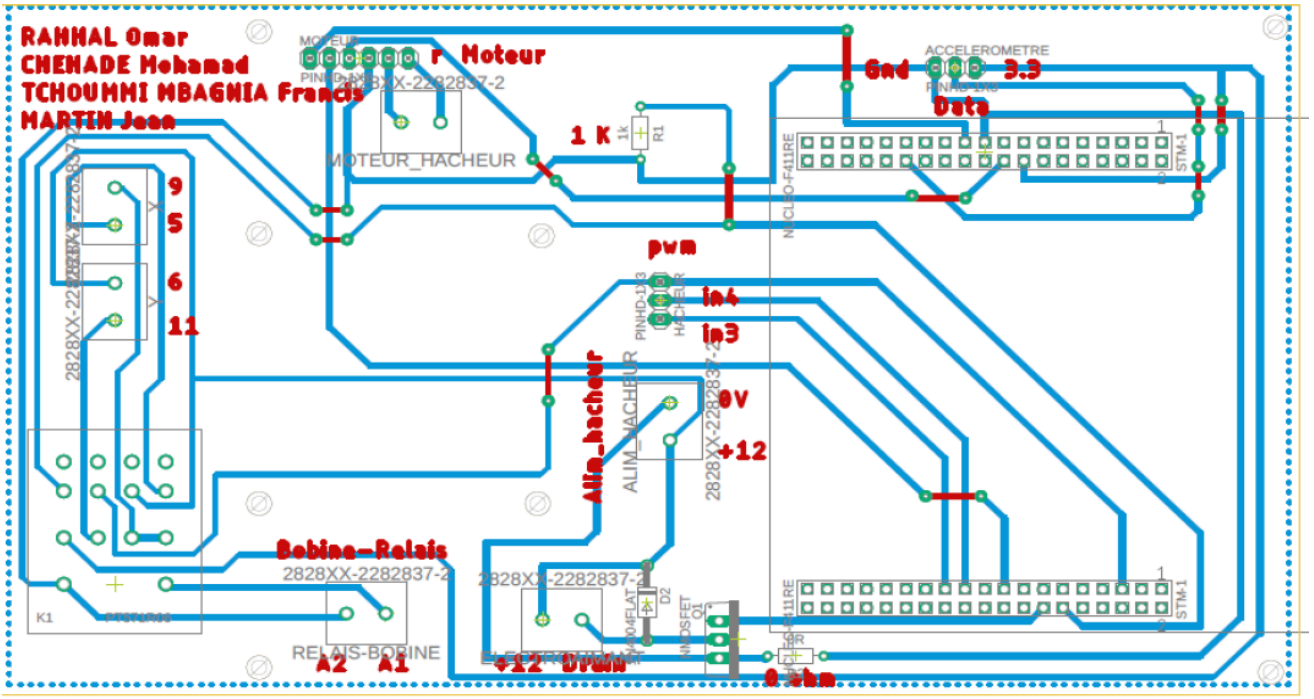
Vérification du câblage, refaire les soudures de l'accéléromètre, changement d'un bornier, vérification de la continuité du courant, ...

Changement de hacheur, car il ne parvenait plus à faire varier la tension pouvant alimenter le moteur. Et le relais de commande ne pouvait non plus rester en position fermée ce qui conduisait à un clignotement de la LED vert qui signifie que le moteur n'est pas alimenté.

Mais après avoir fait le remplacement du hacheur, nous nous sommes rendu compte que le relais du circuit a été endommagé puisqu'il se fermait, mais ne restait pas en position fermée, car le champ magnétique produit était insuffisant pour fermer les contacts du circuit de puissance et par conséquent le moteur ne pouvait pas tourner.

3. Refaire la documentation

Nous avons constaté qu'au niveau du circuit de commande particulièrement le câblage du relais avait des incohérences ne pouvant pas faciliter le câblage de la platine ce qui nous a conduits à refaire un autre circuit EDITSAB afin de corriger l'erreur de nomenclature pouvant faciliter la maintenance de la platine en cas de panne.



Nous pouvons constater sur notre schéma EDITSAB l'alimentation du circuit de commande de notre relais sur la partie sécurité au niveau des contacts du relais normalement ouvert. L'un entre à 12 et ressort normalement ouvert à 8 et l'autre entre à 11 et ressort normalement ouvert à 7.

Ce qui nous permet d'assurer une commande en automatisation de notre circuit pour assurer la sécurité en cas de danger.

Pour le travail, des années précèdent nous avons constaté l'alimentation du circuit de commande du relais sur la partie sécurité au niveau des contacts du relais normalement ouvert donc les entrées était, l'un entre à 12 et ressort normalement ouvert à 5 et l'autre entre à 11 et ressort normalement ouvert à 6, ce qui est impossible sur le câblage d'un relais.

4. Tests

Lancer divers test pour vérifier le bon fonctionnement des composants de la maquette

Afin de mettre la maquette en route, il suffit de la brancher sur le secteur, s'il n'y a pas de problème la led rouge s'allume. Ensuite lors de l'appui du bouton SM (système marche) la led verte s'allume et on entend le relais qui se ferme. La maquette est maintenant alimentée et pilotable depuis l'IHM.

Le bouton SA (système arrêt) permet de stopper l'auto maintien et de couper l'alimentation.

Si jamais cela ne fonctionne pas pour quelconque raison, le bouton d'arrêt d'urgence permet lui aussi d'arrêter la maquette.

5. Dépannages

Mise en place d'un filtre passe-bas d'appoint à la sortie de l'accéléromètre pour réduire les bruits de contacts imparfaits des fils qui apparaissent pendant le mouvement.

Détection des faux contacts des nombreux connecteurs sur la carte, puis soudure sur les points concernés. Par exemple, le bornier de l'électroaimant ou les connexions sur la carte.

Modifications d'appoint pour minimiser les problèmes de masse (toujours présents) afin de ne plus déstabiliser la carte STM32 pendant la mesure.

Modification de l'emplacement des composants et autres éléments pour réduire les problèmes en vibration.

Dépannage du mauvais contrôle de l'aimant en faisant à la main le relâchement.

Détermination des problèmes du hacheur initialement installé (décrochages, faiblesses, faux contacts sur la broche IN3), puis son remplacement par un hacheur différent mais au comportement exactement similaire.

6. Remplacements

Remplacement de composants sur la maquette, comme par exemple l'accéléromètre ou les fils.
Pour les fils, l'objectif a été de réduire leurs longueurs afin de réduire les effets inductifs et les contacts avec la réglette. Cela concerne notamment les fils du PWM et de la direction ainsi que de l'accéléromètre. Ceci est particulièrement utile pour le signal *xout,* afin de réduire le bruit dans la mesure de l'accéléromètre.
Dans le cas de l'accéléromètre, remplacement des capteurs court-circuités par la carte.

Remplacement des supports de capteurs sur la réglette et de leur fixation pour empêcher le mouvement en mesure.

Remplacer des vis pour mieux stabiliser les parties du cadre.

7. Expérimentations

Petits tests sur le remplacement de l'orientation de l'accéléromètre avec comme aboutissement un petit POC sur plaque labdec pour rendre compte de la réduction du bruit et l'augmentation des fréquences.

Modélisation et essais d'un excentrique sur moteur sans réducteur afin de multiplier par 30 le nombre d'excitations possibles. Ceci constitue un POC pour le semestre prochain, principalement dû au manque de temps.

Tests sur transistor MOSFET de puissance pour déterminer pourquoi et comment il s'est détérioré, comment l'utiliser, comment le protéger à l'avenir. Cette micro-tâche est apparue pendant le remplacement du MOSFET.

Nombreux tests sur le remplacement du kas/panel de mise en énergie. Cependant, cela a été abandonné au profit de la tâche N°8 puisqu'il est apparu que le temps était trop court et que le seul moyen d'obtenir un résultat satisfaisant était de refaire la conception.

8. Création d'une carte CAO

Création d'une carte électronique sur eagle pour mettre au propre le filtre passe bas précédemment créé, et intégration de cette carte sur la carte déjà existante.

ÉCOLE NATIONALE D'INGÉNIEURS DE BREST

Tâche N°6

Reprise de la partie Contrôle/Commande et Acquisition

COEFFICIENT DE LA TACHE: 5

Intitulé complet de la tâche:



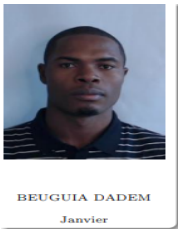
LE CORFF Louis

10%



MADEC Mael

90%



BEUGUIA DADEM Janvier

0%



MEVEL Yoann

0%

1 Changement du code STM32

- Remplacement du code STM32, jugé non maintenable. Le greffon est un code de MCC venu d'IPS.
- Adapter la cartographie (.ioc) du greffon au code original. Les pins sont déjà choisies par la carte imprimée.
- Vérifier le bon fonctionnement. Modifier si nécessaire les parties de codes ne convenant pas au fonctionnement voulu.
- Prévoir des tests unitaires et les indiquer. Créer une petite base de documentation et un repository gitlab pour le maintien et l'historicité du code.

2 Concevoir le protocole de communication avec la STM32

Développer, sur base de l'UART, le protocole de communication entre la STM32 en C et le PC, en python. Puis traiter les données reçues afin de pouvoir les exploiter en post-traitement.

3 Reprise de la partie Contrôle/Commande et Acquisition

- Optimiser la quantité de donnée transmise
- Définir des procédures pour chaque flux de données afin d'en optimiser la qualité et la précision dans un laps de temps réduit.
- Traiter ces flux pour les fiabiliser et les préparer à la transmission.
- Mettre en place la commande du moteur et de l'aimant.

1.1. Analyse fonctionnelle de la tâche

Le code fourni présente des difficultés majeures : il est difficile à maintenir et à comprendre, et son évolution est complexe. Chaque ajustement requis pour la sélection des données ou la modification des paramètres représente un défi. De plus, l'absence de documentation complique davantage la tâche de développement. En outre, des bugs se manifestent de façon récurrente, sans qu'une solution semble être envisageable dans le temps imparti au projet.

Le développement du projet de CMV s'appuie sur Moteur à Courant Continu d'IPS dû à leurs grandes similarités. Pour garantir la qualité du projet en termes de code, l'IPS sert de base, principalement avec les

retours notés de M. Bourgeot. Cette approche nous permet de doubler le temps de code possible tout en restant dans le temps imparti.

Dans le cadre de ce projet, un protocole de communication des données est en cours de développement, tirant parti des fonctionnalités de la bibliothèque pySerial pour la communication, ainsi que de Jupyter Notebook pour sa composante interactive.

Le processus débute par la finalisation d'un projet d'IPS, qui est ensuite décliné vers un nouveau projet. Ce nouveau projet est ensuite adapté aux choix préalablement effectués sur la carte imprimé jaune (pins et routages notamment). Par la suite, une phase de tests est entreprise pour évaluer la réussite de l'adaptation

1.2. Conceptions

Méthodologie de conception, description précise de la solution.

L'objectif global est de développer un système robuste, maintenable et facilement testable, capable de fonctionner de manière simplifiée tout en étant portable grâce à une bonne abstraction.

Choix de conceptions :

Pour garantir la maintenabilité du code, nous adoptons une approche de découpage en une multitude de fichiers. Cette stratégie améliore la lisibilité du code et facilite sa maintenance en permettant une localisation plus aisée des fonctionnalités et des modules.

Pour simplifier le fonctionnement du système, nous adoptons le principe "une tâche, une fonction". Chaque tâche ou responsabilité est attribuée à une fonction distincte, favorisant ainsi la clarté et la compréhension du code.

Afin de simplifier la communication matérielle, nous limitons celle-ci à une ou deux fonctions. En réduisant le nombre de points d'interaction matérielle, nous simplifions le code et réduisons les risques d'erreurs de communication.

Pour atteindre des performances de 1 ms, nous optons pour une stratégie axée sur la performance en utilisant le Direct Memory Access (DMA). Cette approche permet d'optimiser les transferts de données en contournant le processeur principal, ce qui garantit des temps de réponse rapides et une réactivité accrue du système.

Enfin, pour assurer une documentation exhaustive, nous intégrons des commentaires directement dans le code source. Cette pratique permet aux développeurs de comprendre rapidement le fonctionnement de chaque partie du code et facilite la maintenance en fournissant des indications contextuelles sur son utilisation et sa logique interne.

1.3. Bilan des résultats par rapport aux objectifs fixés

Testées, les modifications du projet fonctionnent et permettent comme l'ancien projet mais cette fois-ci fonctionnel, d'interfacer le projet.

1.4. Perspectives d'évolution

Peu de perspectives s'offrent à cette tâche. L'objectif principal d'interfacer la maquette avec un ordinateur est réalisé et aucune erreur relevant du code n'est apparue.

La modularité du code permet d'ajouter des fonctionnalités supplémentaires sans se soucier de maintenir le bon fonctionnement du projet. Une attention est cependant à porter au workspace : sa modification sur la durée le rend non fonctionnel passé un certain point, il faut alors créer un nouveau workspace et ré-importer le projet précédent.

2.1. Conception générale

Estimer le taux de charge du moteur en fonction du modèle numérique électrodynamique pour le scénario de fonctionnement le plus défavorable.

La communication entre la carte d'acquisition et le PC est essentielle. Celle-ci repose sur l'UART (et non l'USART pour des raisons de réception côté STM32) mais aucun protocole ou organisation n'est fourni par STM32CubeIDE ou Python. Il faut alors le concevoir.
Nous avons fait les choix suivants :

- Chaque donnée est précédée d'un identifiant en hexadécimal pour le reconnaître.
- La précision des *float* est limitée à 3.
- Le temps de l'acquisition est ajouté aux données.
- Le baudrate est de 926200 baud/s, pour passer la durée totale d'émission à moins d'une seconde par ligne.
- Les informations sont regroupées, en plus de leurs identifiants, en une trame unique.
- L'envoi est géré par le DMA de la carte.
- TX / RX de chaque côté.
- La STM32 se concentre sur l'émission, le PC sur la réception.
- La trame n'a pas de taille prédéfinie, celle-ci est dynamique.
- La STM32 émet en continu.

2.2. La communication côté PC

Implémenter un correcteur PID et ajuster ses paramètres pour garantir un fonctionnement optimal du système, notamment en termes de consigne de vitesse.

Le langage choisi est le python, du fait des programmes de M. Helbert. C'est sous la forme de Jupyter Notebook que la communication est faite sur le PC, en grande partie puisque nous pouvons grâce à sa forme choisir **quand** écouter quand on veut.

La bibliothèque est **pySerial**, une bibliothèque simple sur laquelle repose nos outils, dont le principal est readline(), qui attend un \n pour arrêter de lire. Celui-ci est le plus fiable parmi les outils et permet surtout de faire une lecture dynamique. Les données reçues étant sous la forme d'un long *string*, elles sont ensuite traitées et séparées pour post-traitement ultérieur.

Côté TX, c'est les objets de type *bytes* (https://docs.python.org/3/library/stdtypes.html#bytes) qui constituent la base de notre envoi. La fonction define_trame() sert de base à la communication, de même que serial.write().

Une documentation pour comprendre et utiliser les outils déjà produits est pourvue.

2.3. La communication côté PC

La STM32 se concentre sur l'émission périodique des informations. Pour des raisons de temps, l'émission de tableaux, plus économique pour le processeur et donc permettant théoriquement de mesurer avec une plus grande fréquence d'échantillonnage, n'est pas implémentée.

Le DMA se charge d'envoyer les informations à travers l'UART. Du fait de la rareté des réceptions, le côté RX est géré par interruption. Divers outils ont été développés pour gérer à la fois la réception et l'émission de trames dynamiques, le tout sous 1ms.

2.4. Modifications et difficultés

Le groupe a rencontré de nombreuses difficultés sur cette tâche :

- La durée d'émission sur le baudrate de base de 115200 baud/s est de 7ms, ce qui rend impossible de prendre des mesures fiables de plus de 50 Hz (Shannon). Nous avons donc multiplié par 8 le baudrate pour passer sous la barre des 1ms de durée d'émission.
- L'allocation dynamique de trames ne survit pas à beaucoup de changements de .metadata. Pour y remédier, nous avons recréé de nombreuses fois des projets.
- De nombreux SEGFAULT non déclarés de l'édition de *string* en C ont caché les problèmes. Seul beaucoup de temps en debug a permis de rendre fonctionnel ces fonctions.

3.1. Prendre compte de l'existant

2 mesures sont à réaliser : l'accéléromètre et l'encodeur.
Des pistes sont réalisées sur la carte afin d'aider à la captation. Cependant, celle de l'accéléromètre produit des appels de courant qui, au mieux redémarre la carte d'acquisition, au pire court-circuite le composant même. Les pistes du moteur sont correctes.

2 actionneurs sont à contrôler : un électroaimant linéaire et un moteur.
L'électroaimant dépend simplement d'un signal logique issu de la carte STM32 et est donc facile à implémenter. A contrario, le moteur dépend de 3 signaux envoyés à un hacheur: un PWM et deux signaux logiques de position pour le pont en H.

3.2. Acquisition des données de l'accéléromètre

L'accéléromètre choisi par nos prédécesseurs est l'ADXL 335.
C'est un accéléromètre analogique 3.3V, ce qui implique donc l'utilisation de l'ADC présent sur la carte.
La stratégie retenue est celle du DMA, qui remplace le processeur dans le déplacement de la donnée obtenue dans la mémoire. Une petite modification est faite sur le temps de blocage de la mesure, de 0 à 480 coups de cloches, la raison étant la réduction de la fréquence d'appel au processeur pour lancer la copie des données. La fréquence passe de 1,75 MHz (ce qui étouffait le processeur) à 84 kHz.

Niveau manipulation, le choix de se séparer des pistes dédiées à été fait afin de ne plus tuer d'accéléromètres.

3.3. Acquisition des données de l'encodeur

L'encodeur associé au moteur Pololu 37D est un encodeur magnétique 64 comptes par révolution.
Contrairement à ces pairs que l'on peut trouver en projet moteur ou en projet robot, il traduit les révolutions en bout de réducteur (si il est présent).

Pour traiter les données envoyées par l'encodeur, deux channels du Timer 4 sont utilisés en mode "Encoder Mode with TI1 & TI2" pour quadrupler la précision. Un code similaire à celui de M. Kerhoas est employé pour la lecture de la vitesse à partir du timer (https://web.enib.fr/~kerhoas/quadEncoder.html).

3.4. Contrôle de l'électroaimant linéaire

L'électroaimant est un composant simple à contrôler. 12V entre ses bornes et il s'ouvre, 0V et il se ferme. Dès lors, on utilise un MOSFET CMOS pour la commande de son ouverture. Nous réutilisons ici le choix de conception des semestres précédents : une entrée logique de 3.3V (tension d'alimentation des GPIOs de la STM32) s'accroche à la patte "Gate" pour la commande, la deuxième pin de l'aimant est associé au "Drain" et la masse se présente à la "Source", ce qui permet de laisser passer ou non le courant et donc la tension.

3.5. Contrôle du moteur

Le contrôle du moteur est effectué par le biais d'un signal PWM sorti du Timer 11 (oui 11) rentrant dans l'entrée ENABLE de la sortie du hacheur. Sa direction est elle donnée par deux signaux logiques (toujours inversés l'un par rapport à l'autre) qui orientent ensuite ou va le 12V et la masse.

Pour ce qui est du PWM, nous avons fait le choix d'une précision à 0.1 afin d'avoir plus de précision dans le contrôle par asservissement et dans le contrôle de petites valeurs de Duty Cycle.

3.6. Perspectives d'évolution

Pas d'évolutions pour le moteur et l'encodeur.

Pour l'électroaimant :
Il faut fiabiliser les connexions des connecteurs et du transistor, le composant est capricieux et réagit mal aux changements de commande.

Pour l'accéléromètre :
Passer sur un accéléromètre en i2c ayant une meilleure fréquence d'échantillonnage (actuellement 1650 Hz sur X et Y, 550 Hz sur Z).
Refaire la conception des pistes entre la carte et l'accéléromètre.

ÉCOLE NATIONALE D'INGÉNIEURS DE BREST

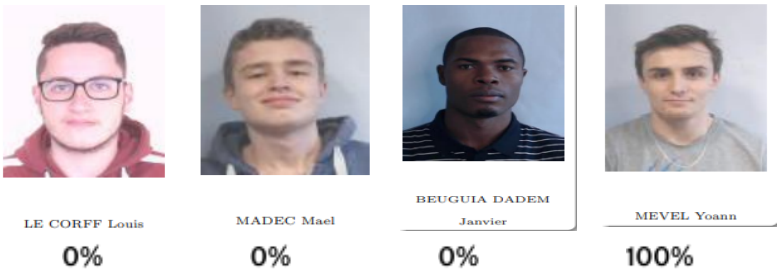
Tâche N°8

Post traitement des résultats expérimentaux puis comparaison avec résultats de simulation.

Intitulé complet de la tâche:

- POST-traitement oscillations forcées (détermination de la pulsation de résonance et mesure du taux d'amortissement à partir du spectre fréquentiel)
- POST-traitement analyse modale (détermination des pulsations modales, des amortissements modaux, ainsi que des déformées modales)
- notices des programmes de post-traitement
- rapport des résultats du post-traitement
- Confrontation avec la théorie

COEFFICIENT DE LA TACHE: 2



LE CORFF Louis	MADEC Mael	BEUGUIA DADEM Janvier	MEVEL Yoann
0%	0%	0%	100%

1. Mise en forme des données pour l'analyse

Dans le cadre de notre projet, après avoir rétabli le bon fonctionnement du système d'acquisition des données, nous collectons les valeurs de l'accéléromètre à la fin de chaque test. Ces données sont ensuite enregistrées dans un fichier Excel, puis importées sous forme de liste dans Jupyter Notebook. Cela nous permet de procéder à une analyse approfondie des données par la suite.

2. Analyse des données avec les notebooks

À l'aide des notebooks utilisés pour la simulation, nous intégrons les données expérimentales afin de procéder à une analyse approfondie des caractéristiques du signal. Cette analyse permet d'identifier et de caractériser les différents modes enregistrés par les capteurs sur la réglette. Le processus implique plusieurs étapes, dont la préparation des données, leur incorporation dans les notebooks, et l'utilisation d'algorithmes spécifiques pour extraire les caractéristiques du signal . Les résultats obtenus offrent une compréhension détaillée des comportements signalés par les capteurs, facilitant ainsi l'interprétation des phénomènes observés et permettant la comparaison avec la simulation.

3. Simulation

Pour simuler le comportement vibratoire de la réglette, nous avons utilisé un signal similaire à une impulsion générée manuellement. Nous avons ensuite remplacé les valeurs d'origine dans le notebook de Mr. Helbert par celles obtenues de notre maquette. Cette modification nous a permis de lancer la simulation des différents modes de vibration de la réglette.

4.Comparaison des résultats expérimentaux avec la simulation

Dans le futur de notre projet, nous effectuerons une comparaison approfondie et une analyse des différences entre les résultats de notre simulation et ceux obtenus à partir du signal réel enregistré. Cette étape nous permettra d'évaluer la précision de notre modèle et d'identifier les écarts éventuels des phénomènes vibratoires observés. En analysant ces différences, nous pourrons affiner notre simulation et ajuster les paramètres pour obtenir des résultats encore plus précis et fiables, contribuant ainsi à l'optimisation des performances de notre système.